

ASSIGNMENT - 1

Aim

To write a C program to perform basic matrix operations such as addition, subtraction, multiplication, and transpose of a matrix using two-dimensional arrays.

Objectives

- To understand the concept of two dimensional array in C
- To perform arithmetic operations on matrices
- To implement matrix addition and subtraction
- To implement matrix multiplication using nested loops.
- To find the transpose of a matrix
- To develop programming skills using arrays and loops.

Theory

A matrix is a rectangular arrangement of elements organized in rows and columns. In C programming, matrices are represented using two-dimensional arrays. Matrix operations are widely used in computer graphics, image processing, machine learning, scientific computing, and engineering applications.

1) **Matrix addition**

Condition: Both matrices must have the same number of rows and columns

$$\text{Formula: } C[i][j] = A[i][j] + B[i][j]$$

2) **Matrix subtraction:**

Condition: Both matrices must have the same order.

$$\text{Formula: } C[i][j] = A[i][j] - B[i][j]$$

3) **Matrix multiplication:**

Condition: Number of columns of the first matrix must be equal to the ~~row~~ number of rows of the second matrix.

$$\text{Formula: } C[i][j] = \sum (A[i][k] \times B[k][j]) \text{ where } k=0 \rightarrow n-1$$

4) **Transpose of a matrix:**

Condition: Obtained by interchanging rows and columns.



Formula: $A[i][j] = A[j][i]$

These operations are implemented using two-dimensional arrays, nested loops, and conditional statements.

Algorithm/pseudocode

Step 1 Start.

Step 2 Create function ~~for~~ to accept values.

```
for (i=0; i<m; i++)
```

```
{
```

```
    for (j=0; j<m; j++)
```

```
    {
```

Accept values

```
    }
```

```
}
```

Step 3 Create display function

```
for (i=0; i<m; i++)
```

```
{
```

```
    for (j=0; j<n; j++)
```

```
    {
```

Display matrix

```
    }
```

```
}
```

Step 4 Create add function

Declare variable

```
for (i=0; i<m; i++)
```

```
{
```

```
    for (j=0; j<n; j++)
```

```
    {
```

$ans[i][j] = a[i][j] + b[i][j]$



Step 5 ~~Ex~~ Create subtraction function

```
for(i=0; i<m; i++)
{
    for(j=0; j<n; j++)
    {
        ans[i][j] = a[i][j] + b[i][j];
    }
}
```

Step 6 Create multiplication function

```
for(i=0; i<m; i++)
{
    for(j=0; j<n; j++)
    {
        ans[i][j] = a[i][j] + b[i][j];
    }
}
```

Step 7 Create a ~~transpose~~ pose function

```
for(i=0; i<m; i++)
{
    for(j=0; j<n; j++)
    {
        a[i][j] Display the switched positions.
    }
}
```

Step 8 Declare main function

Step 9 Declare variables

Step 10 Accept values

Step 11 Call all functions

Step 12 Switch(ch)

case 1: call addition

case 2: call subtraction



case 3: call multiplication function

case 4: call transpose function

default: display "wrong input"

Step 13

Display result matrix

Step 14

Stop.

FAQs 1) Explain matrix addition, subtraction, multiplication, and transpose with suitable examples and state the conditions required for each operation.

- Matrix addition is done only if the matrices are of the same order ~~is done~~ and is done element by element.
- Matrix subtraction has the condition that both the matrices must be of same order and it is done element by element.
- Matrix multiplication is done if no. of columns of the first matrix is equal to the number of columns of the second.
- Matrix transpose is the process in which the position of the ~~column~~ rows of a matrix are portrayed or stored as the column of another.

2) Describe how 2D arrays are used to represent matrices in C programming. Explain the process of accessing and manipulating matrix elements using nested loops.

The two-dimensional arrays enable to represent rows and columns as a grid of elements showing a matrix. In the first loop, we use the integer i , used for the indexing of rows. The second loop uses the variable j used for the indexing of number of columns. Using these loops, we can show or manipulate the elements of a matrix.



3) Write the algorithm for performing matrix multiplication and explain why compatibility between matrices is necessary before multiplication.

Algorithm:

- Step 1 Start
- Step 2 Declare all variables
- Step 3 Accept values of matrix
- Step 4 Check whether no. of columns matches no. of rows.
- Step 5 Using nested loops, $C[i][j] = \sum (A[i][k] \times B[k][j])$
- Step 6 Using nested loops, print the new matrix
- Step 7 Stop.

The compatibility of the matrices is essential because the multiplication is not element by element but dot product.

4) Discuss the applications of matrix operations in real-world domains such as computer graphics, image processing, machine learning, engineering, and scientific computing.

- In computer graphics, matrices are used by transformation matrices to move, rotate or ~~size~~ objects (collection of vertices in 3D)
- In image processing, a digital image is fundamentally a matrix. A 2D matrix portray a greyscale image where every element represents pixel brightness.
- Machine learning uses matrices to arrange real world data, like pixels or spreadsheets, into numerical tables. By multiplying these data tables, AI makes predictions.
- In engineering and scientific computing, matrices play crucial role since they are used for vectors, cross product, etc.



- 5) Explain the role of nested loops in implementing matrix operations inc. Illustrate how nested loops are used for addition, multiplication, and transpose of matrices.

Matrices have two dimensions, therefore nested loops are required to target specific row (i) and columns (j) intersections. For addition, two loops step through the matrices to add the corresponding elements. For transpose, two loops swap the co-ordinates, moving elements $[i][j]$ to $[j][i]$. For multiplication, the first loop ~~is~~ travels through the rows, and the second through the columns and performs the arithmetic.

16/7/26



Code:

```
#include <stdio.h>
```

```
void accept(int a[10][10], int m, int n)
{
    int i, j;

    for (i = 0; i < m; i++)
    {
        for (j = 0; j < n; j++)
        {
            printf("Enter element [%d][%d]: ", i, j);
            scanf("%d", &a[i][j]);
        }
    }
}
```

```
void display(int a[10][10], int m, int n)
{
    int i, j;

    printf("\n");
    for (i = 0; i < m; i++)
    {
        for (j = 0; j < n; j++)
        {
            printf("%d\t", a[i][j]);
        }
        printf("\n");
    }
}
```

```
void addition(int a[10][10], int b[10][10], int m, int n)
{
    int c[10][10];
    int i, j;

    for (i = 0; i < m; i++)
    {
        for (j = 0; j < n; j++)
        {
            c[i][j] = a[i][j] + b[i][j];
        }
    }

    printf("\nAddition of matrices:\n");
    display(c, m, n);
}
```

```
void subtraction(int a[10][10], int b[10][10], int m, int n)
{
    int c[10][10];
    int i, j;

    for (i = 0; i < m; i++)
    {
        for (j = 0; j < n; j++)
        {
            c[i][j] = a[i][j] - b[i][j];
        }
    }

    printf("\nSubtraction of matrices:\n");
    display(c, m, n);
}
```

```

void multiplication(int a[10][10], int b[10][10], int m, int n)
{
    int c[10][10];
    int i, j, k;

    for (i = 0; i < m; i++)
    {
        for (j = 0; j < n; j++)
        {
            c[i][j] = 0;

            for (k = 0; k < n; k++)
            {
                c[i][j] += a[i][k] * b[k][j];
            }
        }
    }

    printf("\nMatrix Multiplication:\n");
    display(c, m, n);
}

void division(int a[10][10], int b[10][10], int m, int n)
{
    int c[10][10];
    int i, j;

    for (i = 0; i < m; i++)
    {
        for (j = 0; j < n; j++)
        {
            if (b[i][j] == 0)
            {
                printf("\nDivision by zero is not possible.\n");
                return;
            }

            c[i][j] = a[i][j] / b[i][j];
        }
    }

    printf("\nElement-wise Division:\n");
    display(c, m, n);
}

void transpose(int a[10][10], int m, int n)
{
    int t[10][10];
    int i, j;

    for (i = 0; i < m; i++)
    {
        for (j = 0; j < n; j++)
        {
            t[j][i] = a[i][j];
        }
    }

    printf("\nTranspose of Matrix A:\n");
    display(t, n, m);
}

int main()
{
    int a[10][10], b[10][10];
    int m, n, ch;

    printf("Enter number of rows and columns: ");
    scanf("%d %d", &m, &n);

    printf("\nEnter Matrix A:\n");
    accept(a, m, n);

    printf("\nEnter Matrix B:\n");
    accept(b, m, n);

    printf("\nMatrix A:\n");
    display(a, m, n);

    printf("\nMatrix B:\n");
    display(b, m, n);

    printf("\nMenu\n");
    printf("1. Addition\n");
    printf("2. Subtraction\n");
    printf("3. Division (Element-wise)\n");
    printf("4. Multiplication\n");
    printf("5. Transpose of Matrix A\n");
    printf("Enter your choice: ");
    scanf("%d", &ch);
}

```

```
scanf("%d", &ch);
```

```
switch (ch)
{
    case 1:
        addition(a, b, m, n);
        break;

    case 2:
        subtraction(a, b, m, n);
        break;

    case 3:
        division(a, b, m, n);
        break;

    case 4:
        if (m == n)
        {
            multiplication(a, b, m, n);
        }
        else
        {
            printf("\nMatrix multiplication requires compatible dimensions.\n");
        }
        break;

    case 5:
        transpose(a, m, n);
        break;

    default:
        printf("\nInvalid choice.\n");
}

return 0;
}
```

Output

```
riamudhole@Ubuntu:~/Desktop/DS Sem 3$ gcc 1691as.c
riamudhole@Ubuntu:~/Desktop/DS Sem 3$ ./a.out
Enter number of rows and columns: 2
2

Enter Matrix A:
Enter element [0][0]: 1
Enter element [0][1]: 2
Enter element [1][0]: 3
Enter element [1][1]: 4

Enter Matrix B:
Enter element [0][0]:
5
Enter element [0][1]: 6
Enter element [1][0]: 7
Enter element [1][1]: 8

Matrix A:
1      2
3      4

Matrix B:
5      6
7      8

Menu
1. Addition
2. Subtraction
3. Division (Element-wise)
4. Multiplication
5. Transpose of Matrix A
Enter your choice: 1

Addition of matrices:
6      8
10     12
```

```
riamudhole@Ubuntu:~/Desktop/DS Sem 3$ ./a.out
Enter number of rows and columns: 2
2

Enter Matrix A:
Enter element [0][0]: 1
Enter element [0][1]: 4
Enter element [1][0]: 764
Enter element [1][1]: 7

Enter Matrix B:
Enter element [0][0]: 2
Enter element [0][1]: 54
Enter element [1][0]: 7
Enter element [1][1]: 3

Matrix A:
1      4
764    7

Matrix B:
2      54
7      3

Menu
1. Addition
2. Subtraction
3. Division (Element-wise)
4. Multiplication
5. Transpose of Matrix A
Enter your choice: 4

Matrix Multiplication:
30     66
1577   41277
```

```
riamudhole@Ubuntu:~/Desktop/DS Sem 3$ ./a.out
Enter number of rows and columns: 2
2

Enter Matrix A:
Enter element [0][0]: 18
Enter element [0][1]: 4
Enter element [1][0]: 27
Enter element [1][1]: 4

Enter Matrix B:
Enter element [0][0]: 2
Enter element [0][1]: 6
Enter element [1][0]: 8
Enter element [1][1]: 2

Matrix A:
18     4
27     4

Matrix B:
2      6
8      2

Menu
1. Addition
2. Subtraction
3. Division (Element-wise)
4. Multiplication
5. Transpose of Matrix A
Enter your choice: 5

Transpose of Matrix A:
18     27
4      4
```

```
riamudhole@Ubuntu:~/Desktop/DS Sem 3$ ./a.out
Enter number of rows and columns: 2
2

Enter Matrix A:
Enter element [0][0]: 2
Enter element [0][1]: 3
Enter element [1][0]: 5
Enter element [1][1]: 7

Enter Matrix B:
Enter element [0][0]: 1
Enter element [0][1]: 5
Enter element [1][0]: 7
Enter element [1][1]: 2

Matrix A:
2      3
5      7

Matrix B:
1      5
7      2

Menu
1. Addition
2. Subtraction
3. Division (Element-wise)
4. Multiplication
5. Transpose of Matrix A
Enter your choice: 3

Element-wise Division:
2      0
0      3
```

```
riamudhole@Ubuntu:~/Desktop/DS Sem 3$ ./a.out
Enter number of rows and columns: 2
2

Enter Matrix A:
Enter element [0][0]: 2
Enter element [0][1]: 3
Enter element [1][0]: 4
Enter element [1][1]: 2

Enter Matrix B:
Enter element [0][0]: 4
Enter element [0][1]: 1
Enter element [1][0]: 3
Enter element [1][1]: 6

Matrix A:
2      3
4      2

Matrix B:
4      1
3      6

Menu
1. Addition
2. Subtraction
3. Division (Element-wise)
4. Multiplication
5. Transpose of Matrix A
Enter your choice: 2

Subtraction of matrices:
-2     2
1      -4
```